



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G1

System Integration and Architecture

System Design Document (SDD)

Project Title: Campus Bites

Prepared By: Policios, Andre C.

Version: 0.1

Date: 02/28/2026

Status: Final

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
0.1	02/21/2026	Andre Policios	Initial draft	Final
0.2	02/28/2026	Andre Policios	Added API specifications	Final
0.3	02/28/2026	Andre Policios	Updated database design	Final
0.4	02/28/2026	Andre Policios	Added UI/UX designs	Final
0.5	02/28/2026	Andre Policios	Incorporated feedback	Final
0.6	02/28/2026	Andre Policios	Final review and corrections	Final
1	02/28/2026	Andre Policios	Baseline version for development	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

Campus Bites is a simplified campus-based food ordering marketplace designed to allow students and faculty to browse food products, add items to a cart, and place orders within the campus environment.

The system consists of:

- Spring Boot backend API
- React Web Application
- Android Mobile Application
- MongoDB database

The goal is to provide a fast, simple, and secure ordering platform tailored specifically for campus users.

1.2 Objectives

1. Develop a fully functional e-commerce MVP with authentication, product catalog, shopping cart, and checkout
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile)
3. Create RESTful APIs for communication between all system components
4. Design a responsive user interface that works consistently across web and mobile platforms
5. Deploy all system components to production-ready environments

1.3 Scope

Included Features:

- User registration and authentication (email/password)
- Product listing with search functionality
- Shopping cart management (add, remove, update quantities)
- Checkout process with shipping information collection
- Order placement with confirmation
- Admin panel for product management

- Responsive web interface
- Native Android mobile application
- PostgreSQL database with relational schema

Excluded Features:

- Real payment gateway integration
- Product reviews and ratings system
- Inventory management automation
- Email notification system
- Social media login integration
- Push notifications
- Advanced analytics dashboard

1.0 INTRODUCTION

1.1 Purpose

This System Design Document (SDD) defines the overall system architecture, functional requirements, non-functional requirements, and implementation plan for Campus Bites.

It serves as a reference document for instructors, developers, and stakeholders to ensure alignment during development and deployment.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Campus Bites

Domain: Campus Food Ordering / E-commerce

Primary Users:

1. Customers (Students, Faculty, Staff)
2. Administrators (Canteen Managers / System Admins)

Problem Statement: Students and campus staff need a fast and convenient way to browse food options and place orders without physically lining up at food stalls. Manual ordering causes long queues, delays, and inefficient order management.

Solution: A minimalist food delivery app focusing on core food shopping functionality with consistent experience across web and mobile.

2.2 Core User Journeys

Journey 1: First-time Customer Purchase

1. User visits web application
2. Clicks "Sign Up" and creates account
3. User browses available food items
4. Clicks product to view details
5. User selects item and views details
6. Views cart and proceeds to checkout
7. User confirms order
8. System displays order confirmation

Journey 2: Returning Customer Purchase

1. User logs in with existing credentials
2. User browses or searches for food items
3. User adds multiple items to cart
4. User updates item quantities
5. User proceeds to checkout and confirms order
6. User receives order confirmation

Journey 3: Administrator Product Management

1. Admin logs in with admin credentials
2. Admin navigates to dashboard
3. Admin adds new food items
4. Admin updates price or stock
5. Admin deletes unavailable items
6. Admin views incoming orders
7. Admin updates order status (Food processing, Out for delivery, Delivered)

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. Product catalog (list, detail, search)
3. Food item listing
4. Food item details view
5. Add to cart functionality
6. Update/remove cart items
7. Checkout and order placement
8. Admin product CRUD
9. Order management dashboard

SHOULD HAVE

1. Search functionality
2. Order history
3. Order status tracking
4. Input validation feedback
5. Responsive design

COULD HAVE

1. Favorite food items
2. Sales summary dashboard
3. Filter by category

WON'T HAVE

1. Online payment gateway integration
2. Email notifications
3. Push notifications
4. Social media login
5. Advanced analytics

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password, First Name, Last Name
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
- **Security:** JWT tokens, password hashing with bcrypt

Feature: Product Catalog

- **Screens:** Product Listing, Product Detail
- **Display:** Grid layout, product images, names, prices, category
- **Search:** By product name
- **API Endpoints:** GET /products, GET /products/{id}, GET /products/search
- **Admin Functions:** POST /products, PUT /products/{id}, DELETE /products/{id}

Feature: Shopping Cart

- **Screens:** Cart View
- **Functions:** Add product, remove product, update quantity
- **Persistence:** Database storage per user
- **API Endpoints:** GET /cart, POST /cart/items, PUT /cart/items/{id}, DELETE /cart/items/{id}

Feature: Checkout Process

- **Screens:** Checkout Form, Order Confirmation
- **Data Collected:** Shipping address (name, address, city, zip)
- **Process:** Validate input, create order, clear cart
- **API Endpoints:** POST /orders, GET /orders/{id}

Feature: Admin Panel

- **Screens:** Product Management, Order List
- **Functions:** Add/edit/delete products, view all orders, and their statuses
- **Access Control:** Admin role required
- **API Endpoints:** Admin-prefixed endpoints with role validation

2.5 Acceptance Criteria

AC-1: Successful User Registration

- Given I am a new user
- When I enter valid email and strong password
- And confirm password matches
- And click "Create Account"
- Then my account should be created
- And I should be automatically logged in
- And redirected to the homepage

AC-2: Product Purchase Flow

- Given I am logged in as a customer
- When I add a product to my cart
- And proceed to checkout
- And enter valid shipping information
- And place the order

- Then I should see order confirmation
- And my cart should be empty
- And the order should appear in admin panel

AC-3: Admin Product Management

- Given I am logged in as an administrator
- When I add a new product with valid details
- And save the product
- Then the product should appear in customer product listing
- And be available for purchase

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 3 seconds
- Support 100 concurrent users
- Database queries complete within 500ms

3.2 Security Requirements

- HTTPS for all communications
- JWT token authentication
- Password hashing with bcrypt (salt rounds = 12)
- SQL injection prevention
- XSS protection
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require role verification

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

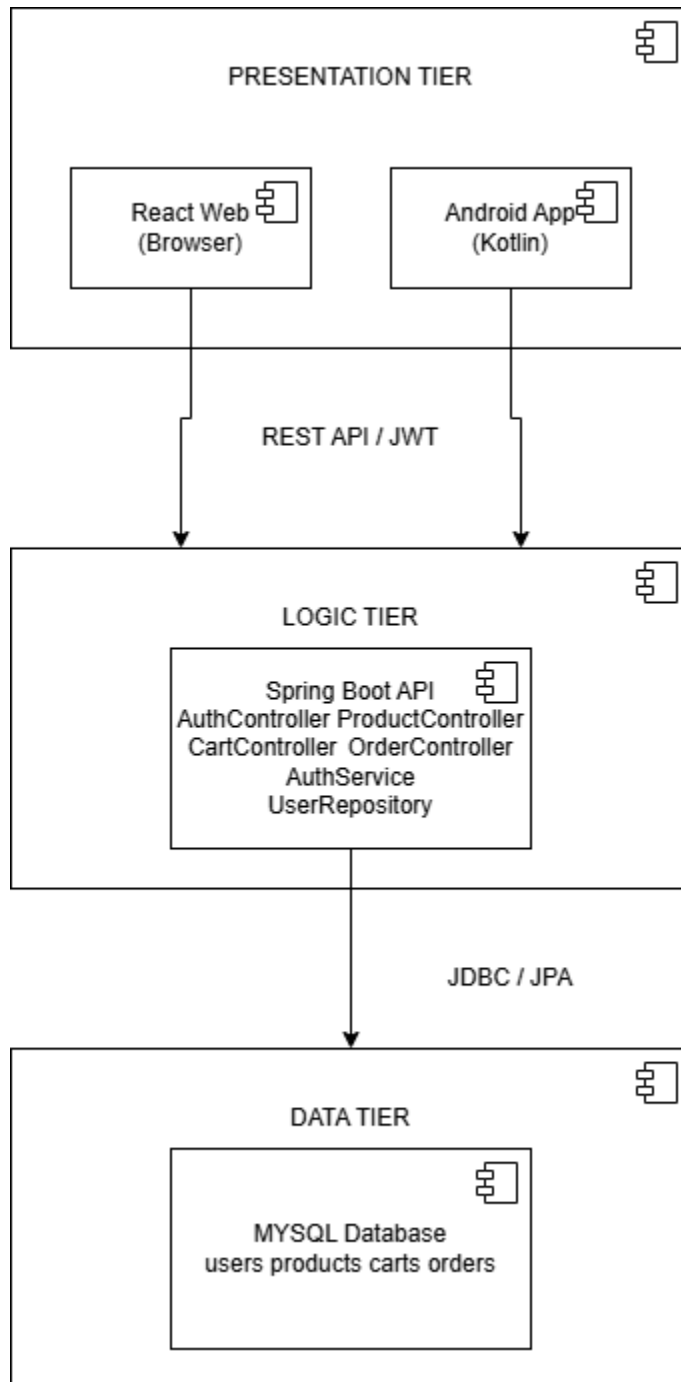
3.4 Usability Requirements

- Complete first purchase within 5 minutes for new users
- WCAG 2.1 Level AA compliance for web
- Consistent navigation across all pages
- Clear error messages with recovery options
- Touch targets minimum 44x44px on mobile

- Keyboard navigation support

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Note: This should be a component diagram

Technology Stack:

- **Backend:** Java 17, Spring Boot 3.5, Spring Security, Spring Data JPA
- **Database:** MySQL 8.0+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://api.campusbites.com/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	<pre>{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": <email>, "password": <password>, "firstname": <firstname>, }</pre>

	<pre> "lastname": <lastname>, } </pre>
Response Structure	<pre> { "success": boolean, "data": { user { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, }, "accessToken": <token>, "refreshToken": <token>, }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string } </pre>

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre> { "email": <email>, "password": <password>, } </pre>
Response Structure	<pre> { "success": boolean, "data": { "user": { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, "role": <role>, }, "accessToken": <token>, "refreshToken": <token>, }, "error": { "code": string, </pre>

	<pre> "message": string, "details": object null }, "timestamp": string } </pre>
--	---

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	<pre> json { "success": false, "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" } } } </pre>

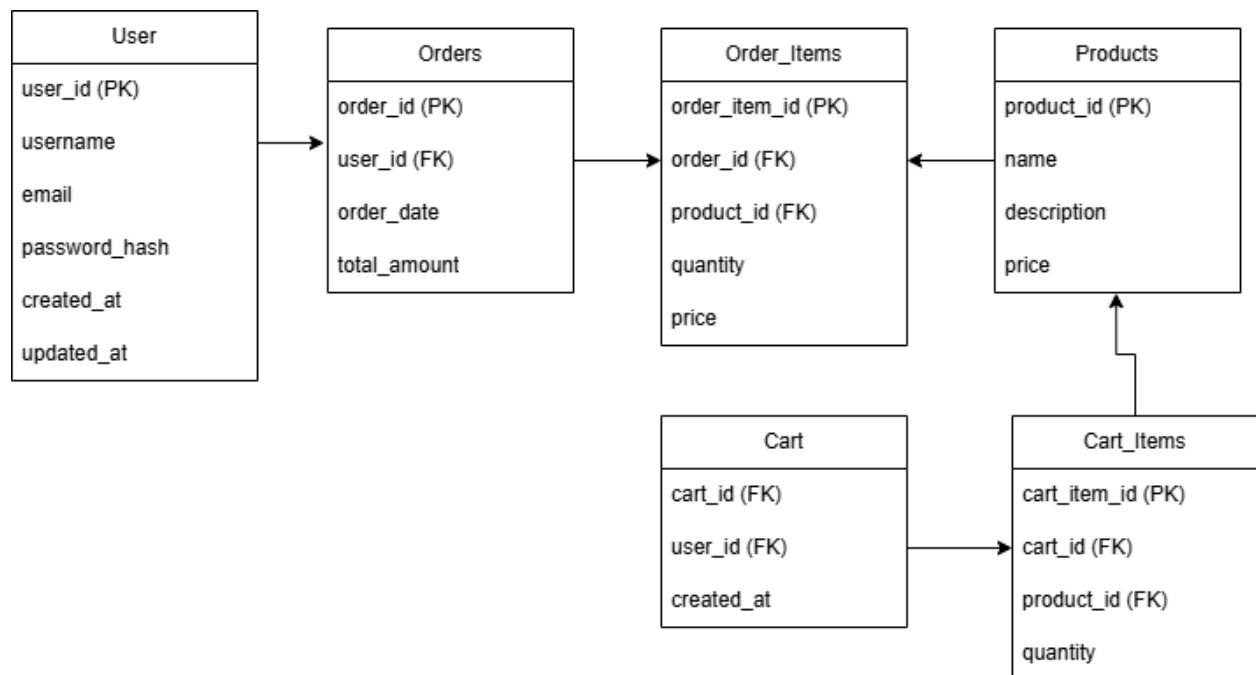
	<pre> }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
--	---

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Note: This should be an ERD (Database Schema)

Detailed Relationships:

Relationship	From	To	Description
One-to-One	User	Carts	Each user has exactly one cart
One-to-Many	User	Orders	A user can place multiple orders
One-to-Many	Carts	Cart_Items	A cart contains multiple items
One-to-Many	Orders	Order_Items	An order contains multiple items
Many-to-One	Cart_Items	Products	Multiple cart items can reference the same product
Many-to-One	Order_Items	Products	Multiple order items can reference the same product

6.3 Key Tables

Table: User

Column	Data Type	Constraint	Description
user_id	INT	PK, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(100)	NOT NULL	Display name of the user
email	VARCHAR(255)	NOT NULL, UNIQUE	User email address
password_hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
created_at	DATETIME	NOT NULL	Account creation timestamp
updated_at	DATETIME	NOT NULL	Last update timestamp

Table: Orders

Column	Data Type	Constraint	Description
order_id	INT	PK, AUTO_INCREMENT	Unique order identifier

Column	Data Type	Constraint	Description
user_id	INT	FK → User.user_id	Reference to the ordering user
order_date	DATETIME	NOT NULL	Date and time order was placed
total_amount	DECIMAL(10,2)	NOT NULL	Total cost of the order

Table: Order_Items

Column	Data Type	Constraint	Description
order_item_id	INT	PK, AUTO_INCREMENT	Unique order item identifier
order_id	INT	FK → Orders.order_id	Reference to the parent order
product_id	INT	FK → Products.product_id	Reference to the product
quantity	INT	NOT NULL	Number of units ordered
price	DECIMAL(10,2)	NOT NULL	Unit price at time of order

Table: Products

Column	Data Type	Constraint	Description
product_id	INT	PK, AUTO_INCREMENT	Unique product identifier
name	VARCHAR(255)	NOT NULL	Name of the product
description	TEXT	NULLABLE	Detailed product description
price	DECIMAL(10,2)	NOT NULL	Current price of the product

Table: Carts

Column	Data Type	Constraint	Description
cart_id	INT	PK, AUTO_INCREMENT	Unique cart identifier
user_id	INT	FK → User.user_id, UNIQUE	Reference to the owning user (one-to-one)
created_at	DATETIME	NOT NULL	Cart creation timestamp

Table: Cart_Items

Column	Data Type	Constraint	Description
cart_item_id	INT	PK, AUTO_INCREMENT	Unique cart item identifier
cart_id	INT	FK → Carts.cart_id	Reference to the parent cart
product_id	INT	FK → Products.product_id	Reference to the product
quantity	INT	NOT NULL	Number of units in the cart

6.4 SQL Schema

```

CREATE TABLE User (
  user_id      INT PRIMARY KEY AUTO_INCREMENT,
  username     VARCHAR(100) NOT NULL,
  email        VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at   DATETIME NOT NULL,
  updated_at   DATETIME NOT NULL
);

CREATE TABLE Products (
  product_id  INT PRIMARY KEY AUTO_INCREMENT,
  name        VARCHAR(255) NOT NULL,
  description  TEXT,
  price       DECIMAL(10,2) NOT NULL
);

CREATE TABLE Carts (

```

```

    cart_id      INT PRIMARY KEY AUTO_INCREMENT,
    user_id      INT NOT NULL UNIQUE,
    created_at   DATETIME NOT NULL,
    FOREIGN KEY (user_id) REFERENCES User(user_id)
);

CREATE TABLE Cart_Items (
    cart_item_id INT PRIMARY KEY AUTO_INCREMENT,
    cart_id      INT NOT NULL,
    product_id   INT NOT NULL,
    quantity     INT NOT NULL,
    FOREIGN KEY (cart_id) REFERENCES Carts(cart_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

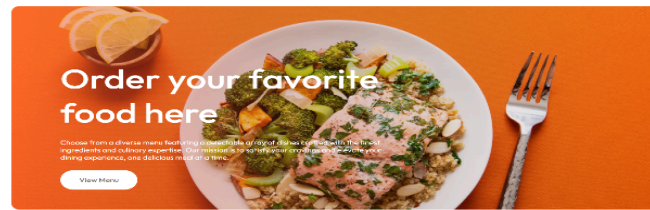
CREATE TABLE Orders (
    order_id     INT PRIMARY KEY AUTO_INCREMENT,
    user_id      INT NOT NULL,
    order_date   DATETIME NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (user_id) REFERENCES User(user_id)
);

CREATE TABLE Order_Items (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id      INT NOT NULL,
    product_id    INT NOT NULL,
    quantity      INT NOT NULL,
    price         DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

```

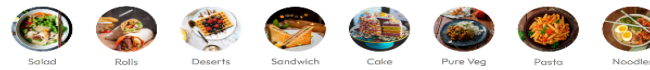
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

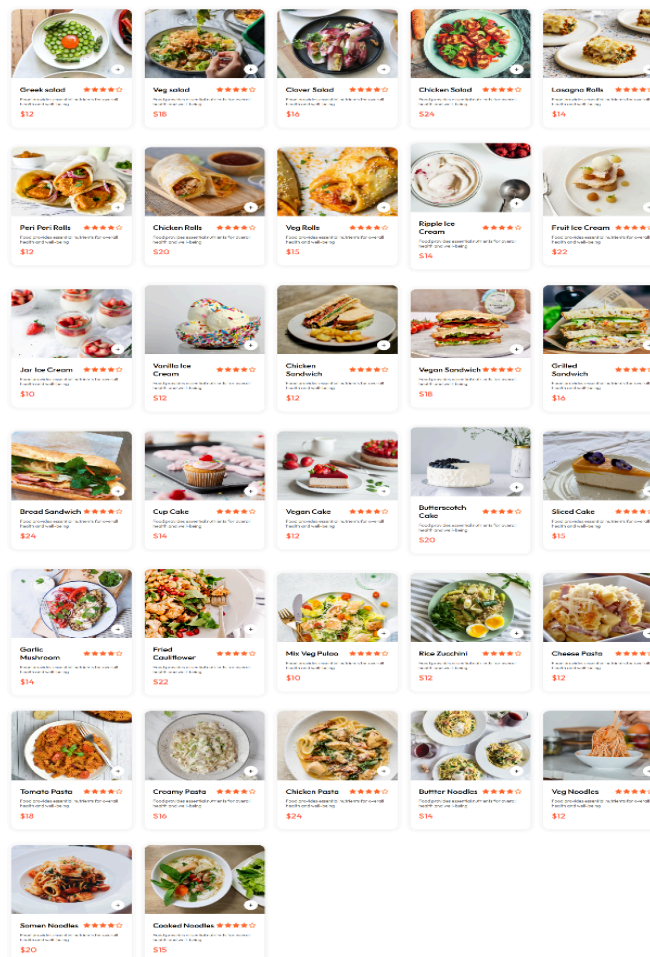


Explore our menu

At Tomato, we believe in fresh, healthy food that is also delicious. We offer a variety of dishes that are both healthy and delicious. Our menu is designed to be easy on the stomach and easy on the wallet. We use only the best ingredients and our chefs are experts in their craft. We are committed to providing a great dining experience, one dish at a time.



Top dishes near you



For Better Experience Download
Tomato App



Tomato.

Tomato is a family owned and operated business. Since 1999, we have been the leading chain of dining fast food in the USA, which is known for its quality of food and customer service. We are committed to providing a great dining experience, one dish at a time. We use only the best ingredients and our chefs are experts in their craft. We are committed to providing a great dining experience, one dish at a time.

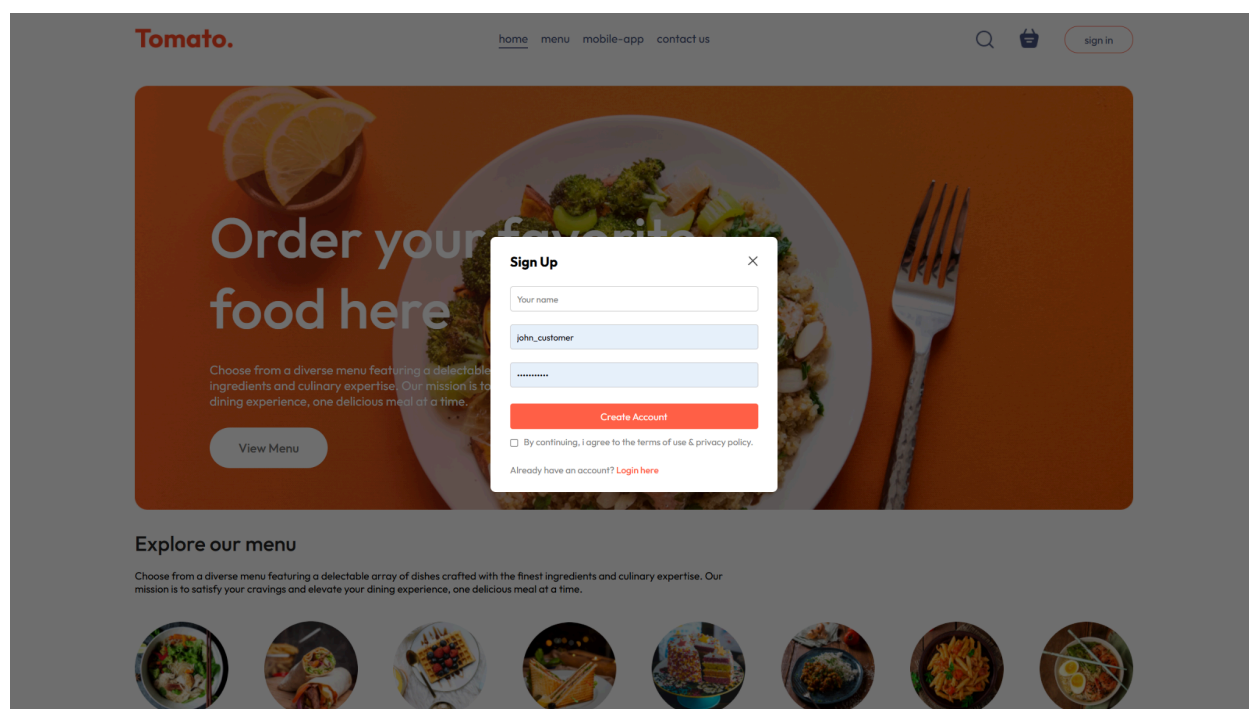
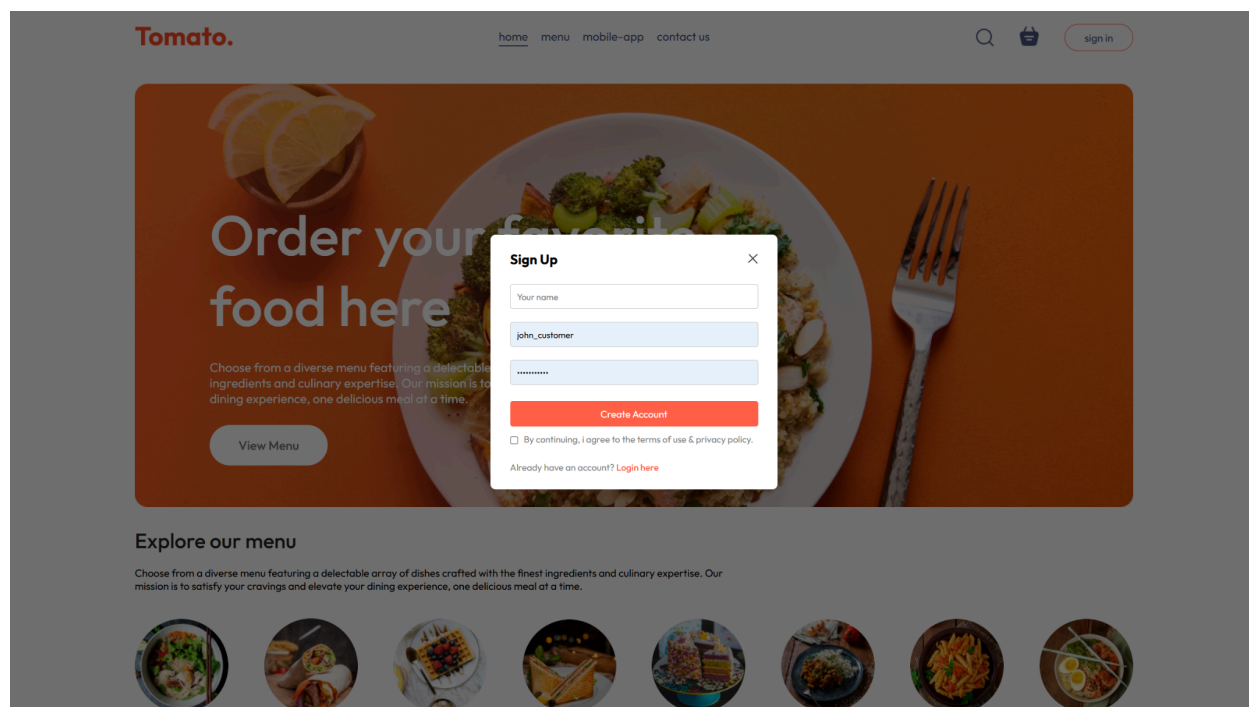


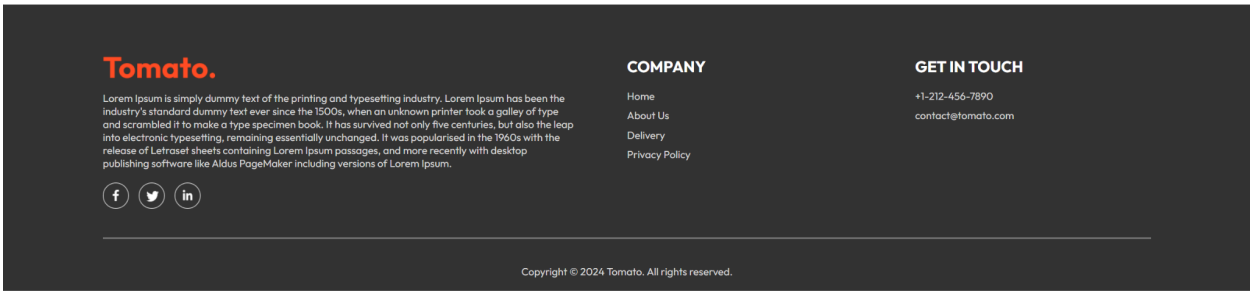
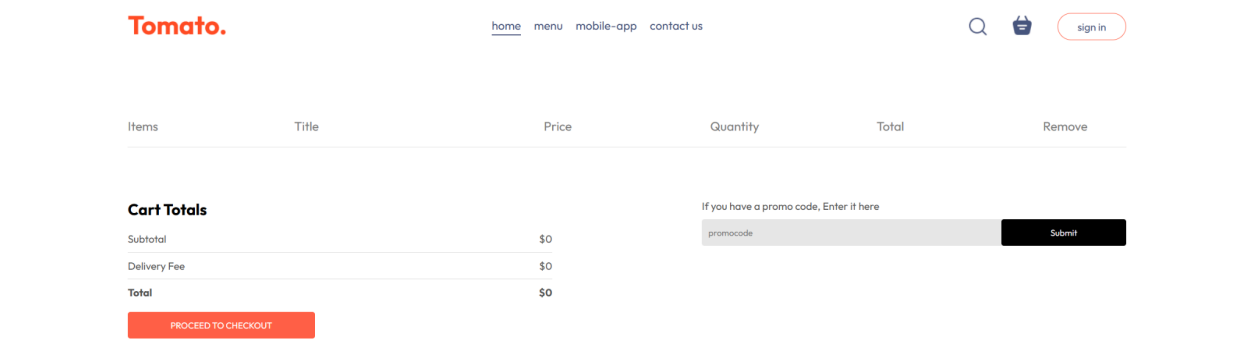
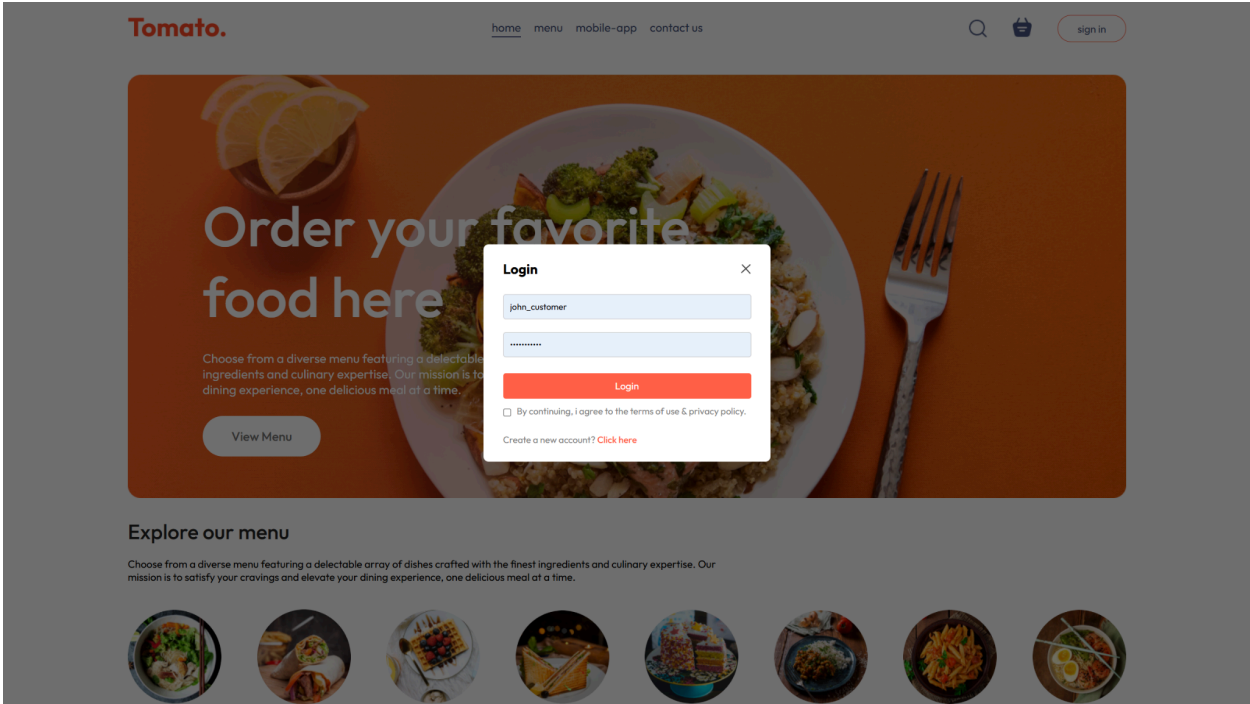
COMPANY

Home
About Us
History
Privacy Policy

GET IN TOUCH

+1 312 656 1890
contact@tomato.com





Tomato.

[home](#) [menu](#) [mobile-app](#) [contact us](#)

sign in

Delivery Information

First name

Last name

Email address

Street

City

State

Zip code

Country

Phone

Cart Totals

Subtotal

\$0

Delivery Fee

\$0

Total

\$0

PROCEED TO PAYMENT

Tomato.

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.

f

t

in

COMPANY

Home

About Us

Delivery

Privacy Policy

GET IN TOUCH

+1-212-456-7890

contact@tomato.com

Copyright © 2024 Tomato. All rights reserved.

Homepage (Product Listing)

Section	Description
Header / Navigation	Logo (top-left), navigation links (Home, Menu, Mobile-App, Contact Us), search icon, cart icon, and Sign In button (top-right)
Hero Banner	Full-width banner image of featured food with overlay text: 'Order your favorite food here', a short tagline, and a 'View Menu' call-to-action button
Category Row	Horizontally scrollable circular category icons (Salad, Rolls, Desserts, Sandwich, Cake, Pure Veg, Pasta, Noodles) with labels below each icon
Product Grid — Top Dishes	Grid layout (4 columns on desktop) showing food cards. Each card contains: food image, dish name, star rating, short description, price, and an Add (+) button
App Download Banner	Promotional section: 'For Better Experience Download Tomato App' with Google Play and App Store badge buttons

Section	Description
Footer	Brand name and description (left), Company links: Home, About Us, Delivery, Privacy Policy (center), Get In Touch: phone and email (right), social media icons, copyright notice

Menu Page (Browse All Products)

Section	Description
Header / Navigation	Same persistent header as homepage
Category Filter	Horizontal row of category icons for filtering products by type (Salad, Rolls, Desserts, Sandwich, Cake, Pure Veg, Pasta, Noodles)
Section Title	"Top dishes near you" heading with a subtitle describing the menu
Product Grid	Responsive grid (4 columns desktop, 2 columns mobile) displaying all available food items with image, name, rating, description, price, and Add (+) button per card
Pagination / Scroll	Continuous scroll loads additional products as the user browses

Product Card Component

Element	Description
Food Image	Rectangular image thumbnail of the dish (top of card)
Add Button	Small circular '+' button overlaid on the bottom-right of the image
Dish Name	Bold text label below the image
Star Rating	4-5 star visual rating indicator in orange
Description	Short two-line description text in gray
Price	Price displayed in orange/red accent color (e.g. \$12)

Login / Register Page

Section	Description
Header	Same persistent navigation header
Login Form	Centered card with fields: Email, Password, and a 'Sign In' button
Register Form	Fields: First Name, Last Name, Email, Password, Confirm Password, and a 'Create Account' button
Toggle Link	Link to switch between Login and Register views
Validation	Inline error messages shown below each field on invalid input

Cart / Checkout Page

Section	Description
Cart Icon	Cart icon in the header shows item count badge when items are added
Cart Table	Table layout with columns: Items, Title, Price, Quantity, Total, and Remove — each added product appears as a row
Promo Code	Right-side panel with a promo code input field and Submit button
Cart Totals Panel	Displays Subtotal, Delivery Fee, and Total with values aligned to the right
Proceed to Checkout Button	Full-width orange 'PROCEED TO CHECKOUT' button below the cart totals
Delivery Information Form	Left panel with input fields: First Name, Last Name, Email Address, Street, City, State, Zip Code, Country, Phone
Cart Summary (Checkout)	Right panel showing Cart Totals: Subtotal, Delivery Fee, Total with a 'PROCEED TO PAYMENT' orange button
Footer	Dark footer with brand description, Company links (Home, About Us, Delivery, Privacy Policy), Get In Touch section (phone, email), social icons, and copyright

7.2 Mobile Application Wireframes

The mobile application mirrors the web interface with touch-optimized interactions and a bottom navigation bar for primary actions.

Screen	Description
Home Screen	Hero banner, horizontal category scroll, and vertical product feed with 2-column grid layout
Menu / Browse Screen	Filterable product grid with category tabs at the top

Screen	Description
Product Detail Screen	Full image, dish name, rating, description, price, quantity selector, and 'Add to Cart' button fixed at the bottom
Cart Screen	List of cart items, quantity controls, order summary, and 'Place Order' button
Checkout Screen	Delivery address form, order review, and 'Confirm Order' button
Profile Screen	User details, order history, and logout option
Bottom Navigation	Icons for Home, Menu, Cart, and Profile

Note: Insert Mobile App wireframe/screenshots here.



7.3 Design System

Property	Value
Primary Color	Tomato Red / Orange (#FF4500) — used for accents, prices, buttons, and logo
Background Color	White (#FFFFFF) for content areas; Dark (#2D2D2D) for footer
Text Color	Dark Gray (#333333) for headings; Gray (#666666) for descriptions
Typography	Sans-serif font family; bold for dish names and prices; regular for descriptions
Card Style	White card with subtle shadow, rounded corners, image on top
Button Style	Rounded pill buttons; orange/red primary; white outlined secondary
Spacing	Consistent 8px grid system for padding and margins
Responsive Breakpoints	Mobile (360px+), Tablet (768px+), Desktop (1024px+)
Touch Targets	Minimum 44x44px for all interactive elements on mobile

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

- Day 1: Spring Boot setup with dependencies
- Day 2: Database configuration and entities
- Day 3: JWT authentication implementation
- Day 4: User management endpoints
- Day 5: Product CRUD operations

Week 4: Core Features

- Day 1: Cart functionality
- Day 2: Order management
- Day 3: Search and filtering
- Day 4: Error handling and validation
- Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

- Day 1: React setup with TypeScript
- Day 2: Authentication pages (login, register)
- Day 3: Product listing page
- Day 4: Product detail page
- Day 5: Shopping cart implementation

Week 6: Complete Web Features

- Day 1: Checkout flow
- Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Product browsing

Day 4: Shopping cart

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow

Day 2: Order management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements